

Product Recognition on Store Shelves

Computer Vision project · University of Bologna

Carl Fredrik Cornelis Nexmark

18 June 2026

1 Introduction

This project was carried out as part of a Computer Vision examination and investigates object detection techniques in supermarket environments. The main objective is to design a system capable of identifying products placed on store shelves through the use of computer vision methods.

Given an image of a supermarket shelf, the system is expected to recognize and localize the products present in the scene.

The specific task addressed in this project is the detection of cereal boxes from different brands using a reference image for each product. For every detected product instance, the system must determine the number of occurrences, estimate the dimensions of the corresponding bounding box in pixels, and calculate the position of the object within the image using the center coordinates of the bounding box.

Object recognition in real-world environments requires methods that can handle variations in viewing conditions, including differences in scale, orientation, and lighting. Local invariant features address these challenges by enabling reliable identification of distinctive image characteristics despite such transformations. A complete recognition framework typically consists of detecting key features, generating descriptive representations, matching corresponding features between images, and estimating object position. Together, these steps form an effective pipeline for detecting and recognizing objects in images and video data.

2 Task A — Multiple Product Detection

Task A works with the easy shelves e1–e5, where each scene contains at most one instance of each product. The product set is $\{0, 1, 11, 19, 24, 25, 26\}$, shown in Figure 1.

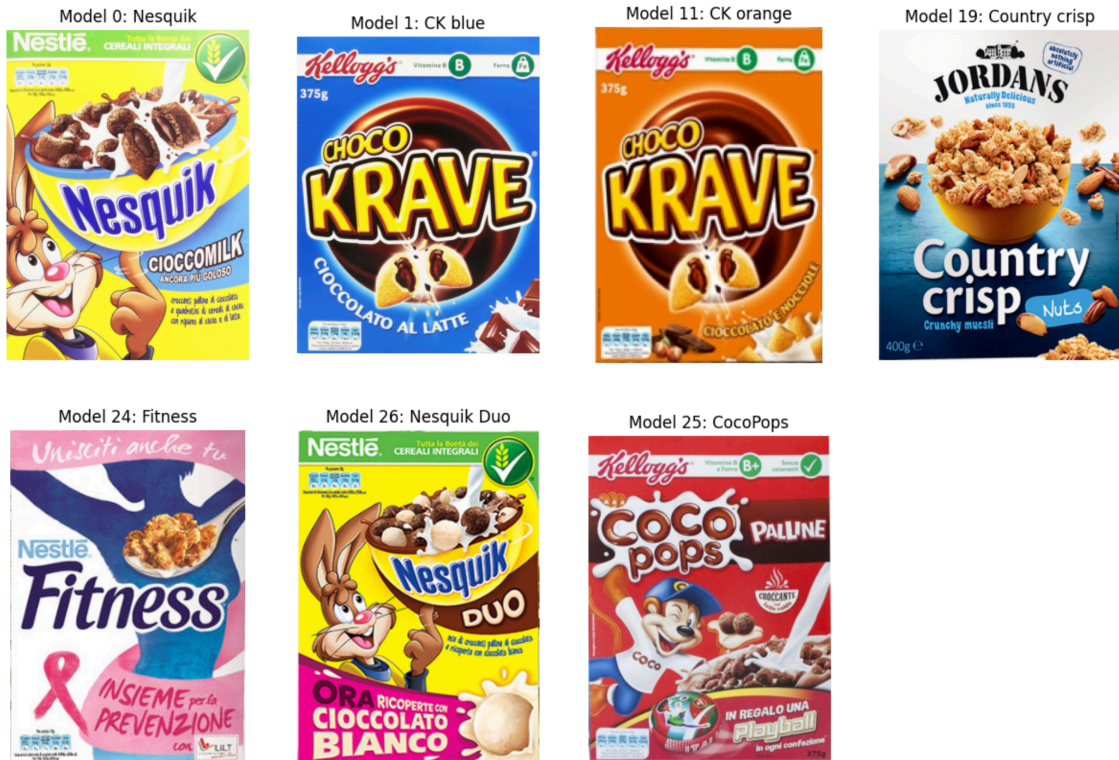


Figure 1: The seven product reference images used in Task A (and reused in Task B).

The pipeline therefore needs to do one thing well. Given a model image and a scene image, decide whether the product is present and, if so, return its bounding box. To solve this, SIFT keypoint matching followed by RANSAC homography estimation was used. This is a classical local-feature pipeline that fits the problem because the printed face of a cereal box is essentially planar.

2.1 Feature extraction and matching

Keypoints and descriptors on both the model and the scene are extracted with SIFT. SIFT detects blob-like keypoints at extrema of the Difference-of-Gaussians scale-space. Each keypoint carries a position, a characteristic scale, and a dominant orientation. Around each keypoint, gradients are sampled from a normalised patch into a 128-D histogram descriptor that is L2-normalised, clipped at 0.2, and re-normalised. This gives invariance to affine intensity changes and robustness to non-linear ones, on top of the rotation invariance provided by the canonical orientation. SIFT was chosen because cereal-box covers are rich in texture and text, which is likely type of content that produces stable, repeatable keypoints, and the descriptor's invariances cover the kinds of pose and lighting variation seen across shelves.

Model keypoints are matched to scene keypoints with FLANN, an approximate nearest-neighbour library. It applies Best-Bin-First search over randomised k-d trees, which is what makes it tractable at the 128-D SIFT descriptor size. For every model descriptor FLANN returns the two closest scene descriptors at distances $d_1 \leq d_2$. Lowe's ratio test then keeps a match only if $d_1 < 0.6 \cdot d_2$, if the second-closest match is comparably close, the first is ambiguous and likely wrong. A threshold value was found at 0.6 from empirical tuning.

If fewer than 40 matches survive the ratio test, the (model, scene) pair is skipped entirely, since RANSAC with very few matches is unlikely to find a good homography and more likely to produce false positives. This threshold is intentionally high, cause when combined with the strict ratio above, it makes the precision/recall trade-off skew well toward precision before RANSAC is even invoked.

2.2 Geometric verification

After the ratio test some matches are still wrong. RANSAC is used to fit a planar homography to the surviving point correspondences. It repeatedly samples four match pairs at random, estimates the homography they imply, and counts inliers, matches whose reprojection error under the estimated homography is below 5 px. After many iterations the homography with the largest inlier set is returned. A homography is the right model here, since each cereal-box face is essentially planar under a perspective camera, so the estimate both yields an accurate bounding box and serves as a geometric *verification* of the matches. A coherent box produces a homography with many consistent supporters.

A detection is accepted only when at least 15 matches are RANSAC inliers. This threshold is the main false-positive guard. When it is met, the four corners of the model image are warped through the estimated homography to obtain an axis-aligned bounding box.

2.3 Colour check

To differentiate between objects with similar design but different colours, such as products 1 and 11, a colour check was added based on the Euclidean distance between the BGR histograms of the model and the detected region in the scene. Other options such as the Bhattacharyya and Mahalanobis distances, mentioned in lecture notes, were considered, but Euclidean distance was chosen for its simplicity and effectiveness. The comparison is performed in the BGR colour space, the default in OpenCV.

The threshold was determined empirically by comparing the Euclidean distances of three model images (0, 1, 11) against the same scene region (Figure 2). The distance for the correct match (1) was significantly lower than for the incorrect ones, and a threshold of 50 separates them cleanly while allowing some variability in BGR mean values across scenes. A worthy mention is product 0 in this test, which got the highest distance, even though it has much of the the same color profile of the scene, cause of the matching product. The reason for this is unknown, and test results were not replicated with several runs on product 0 alone.

Colour verification — mean BGR + Euclidean distance

SIFT runs on grayscale, so it cannot tell apart products that share gradient structure but differ in colour (e.g. P1 blue Krave vs P11 orange Krave). We add a separate check: compare the mean BGR of the model to the mean BGR of the warped region in the scene. If the distance is large, the colour doesn't match — reject the detection.

Because the walkthrough currently runs `model_id = "1"` (blue Krave) against `e1`, the warped region in the scene is the orange Krave box. We test 3 models against that region:

- model `0` (yellow Nesquik) — should give a large distance (correct rejection)
- model `1` (blue Krave) — should give a large distance (correct rejection)
- model `11` (orange Krave) — should give a small distance (correct accepted)

```
def quad_mask(scene_shape, warped):
    m = np.zeros(scene_shape[:2], np.uint8)
    cv2.fillPoly(m, [np.int32(warped)], 255)
    return m

def mean_bgr(img, mask=None):
    return np.array(cv2.mean(img, mask=mask)[:3])

def color_distance(model_img, scene_img, warped):
    m_model = mean_bgr(model_img)
    m_scene = mean_bgr(scene_img, mask=quad_mask(scene_img.shape, warped))
    return float(np.linalg.norm(m_model - m_scene))

model_0_bgr = cv2.imread(str(MODELS_DIR / "0.jpg"), cv2.IMREAD_COLOR)
model_1_bgr = cv2.imread(str(MODELS_DIR / "1.jpg"), cv2.IMREAD_COLOR)
model_11_bgr = cv2.imread(str(MODELS_DIR / "11.jpg"), cv2.IMREAD_COLOR)

print(f"BGR Euclidean distance between model and warped region in {scene_id}:")
print(f" model 0 (yellow Nesquik) vs region : {color_distance(model_0_bgr, scene_bgr, warped):7.2f}")
print(f" model 1 (blue Krave) vs region : {color_distance(model_1_bgr, scene_bgr, warped):7.2f}")
print(f" model 11 (orange Krave) vs region : {color_distance(model_11_bgr, scene_bgr, warped):7.2f}")
```

38] ✓ 0.0s Python

```
... BGR Euclidean distance between model and warped region in e1:
model 0 (yellow Nesquik) vs region : 92.22
model 1 (blue Krave) vs region : 77.56
model 11 (orange Krave) vs region : 40.27
```

Figure 2: Euclidean distance between the BGR histograms of the detected region in the scene and the model images for products 0, 1, and 11.

2.4 Iterative masking sweep

To process a whole shelf it is wrapped in an *iterative masking sweep*. At each iteration the full single-pair pipeline is run for every remaining candidate product, the product with the most RANSAC inliers is accepted, its bounding box is filled with black in a working copy of the scene, that product is removed from the candidate set, and SIFT is recomputed on the masked scene before the next iteration. The loop terminates when no remaining product yields an accepted detection.

This is known as a greedy detection strategy, and it is a simple way to allow multiple products to be detected in the same scene without one product's detections interfering with another's. It is locally optimal, but not globally, a confidently wrong winner can lock out a correct competitor. More principled alternatives exist, which may likely give a better global solution.

2.5 Results

The figures below show the accepted detections on each of the five easy shelves after the iterative masking sweep. Green rectangles mark the warped model corners drawn back onto the original scene.



Figure 3: Task A — accepted detections on shelf e1.



Figure 4: Task A — accepted detections on shelf e2.

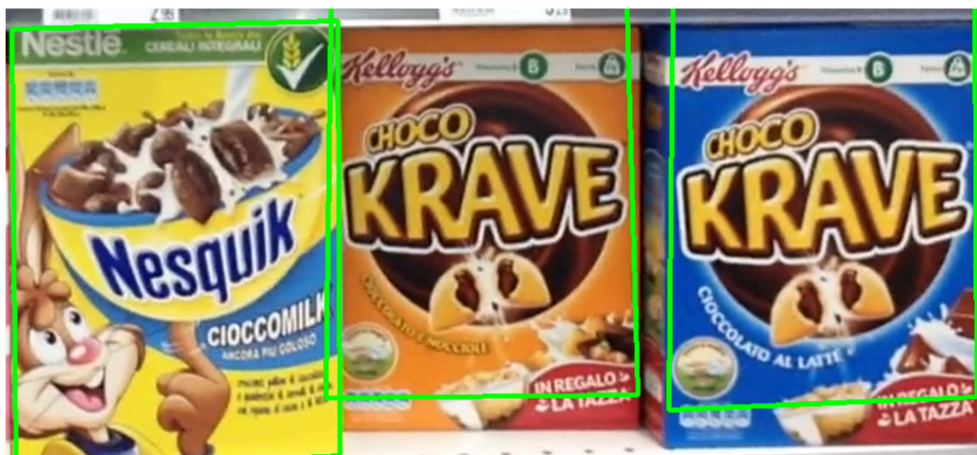


Figure 5: Task A — accepted detections on shelf e3.

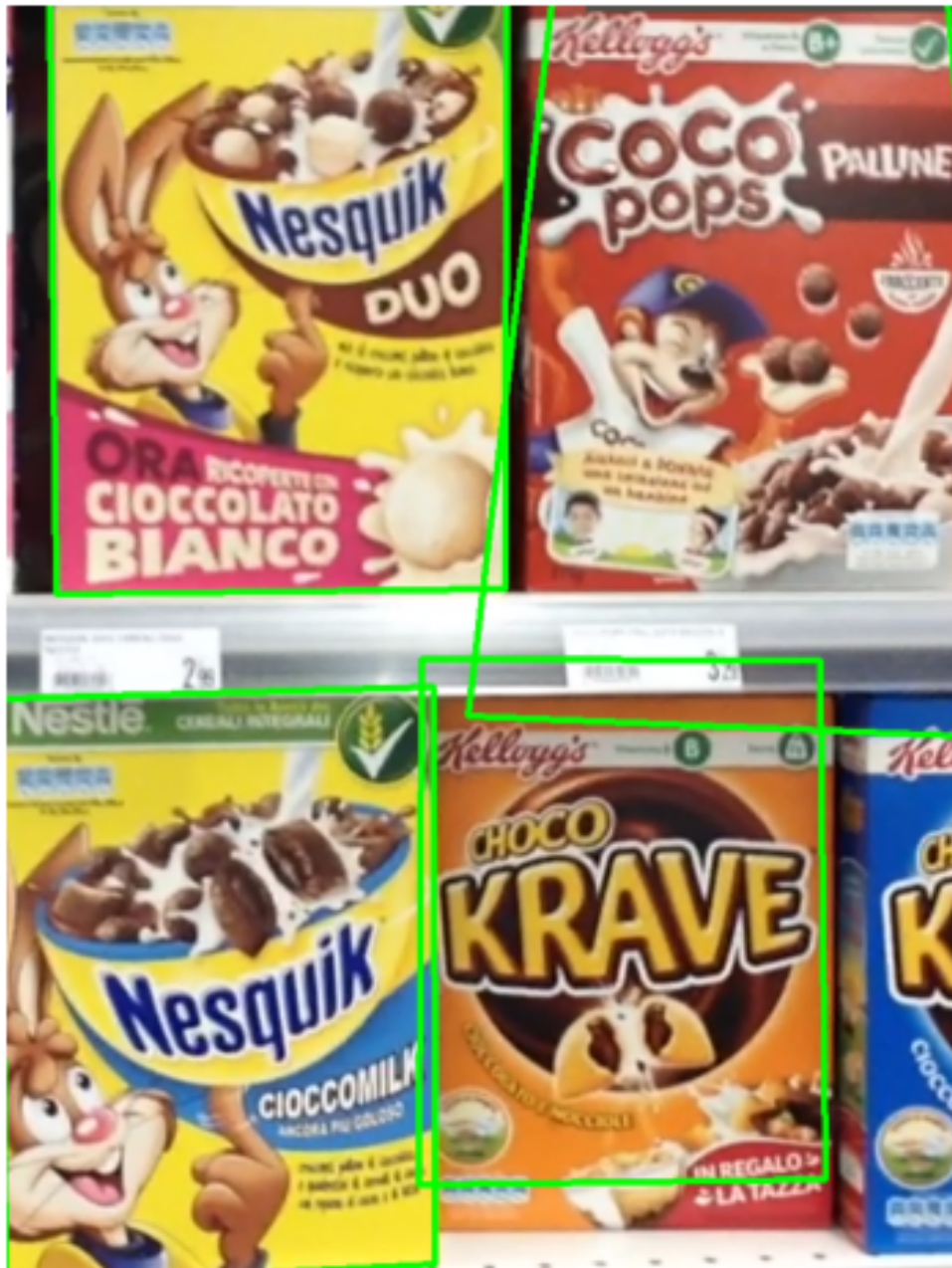


Figure 6: Task A — accepted detections on shelf e4.



Figure 7: Task A — accepted detections on shelf e5.

2.6 Discussion

Known limitations.

The colour check is what separates visually similar products that differ in colour palette, and the cleanest success is on the Krave variants. The BGR-mean distance between the wrong Krave model and the scene region comfortably exceeds the threshold of 50, and the incorrect match is rejected unambiguously. The Nesquik pair — Cioccomilk (product 0) and DUO (product 26) is the counter-example. The two boxes share the Nesquik mascot and a large fraction of the printed graphics, so SIFT descriptors and the fitted homography support both at the same shelf location. The colour palettes are also closer than the threshold accounts for, even though Cioccomilk carries a pink accent that distinguishes it visually, the BGR mean over the full bounding box averages out into something only marginally different from DUO, is well within the 50-unit tolerance. The colour check therefore does *not* separate the two reliably. This is where the iterative masking sweep makes the problem to solve easier, as soon as one of the two Nesquik models is accepted at a shelf location, the region is blacked out, and the second model can no longer fire there.

A different failure mode is observed on the Coco Pops product, where the homography itself goes wrong rather than the colour, seen from scene e4 in Figure 6. The exact reason is not clear, one plausibility is that the line with the price tag confuses the homography estimation, making it leak onto neighbouring boxes.

3 Task B — Multi-instance Detection

Task B works on the medium shelves m1–m5, where each scene can contain multiple instances of the same product. The product set is the same as Task A, {0, 1, 11, 19, 24, 25, 26}. Task A's pipeline is fundamentally limited here, since RANSAC returns at most one homography per pair, so it cannot find more than one instance per product. Task A's SIFT, FLANN, and Lowe's ratio test are kept in Task B. The geometric verification step is replaced with a Generalised Hough Transform that votes for product locations in the scene.

3.1 Voting with a star model

Feature extraction and matching follow Task A in shape but loosen the threshold slightly. SIFT keypoints and descriptors are extracted on the model and the scene, matched with FLANN at $k = 2$, and filtered with Lowe’s ratio test.

For each model keypoint at position p_i a *joining vector* is precomputed from the keypoint to the model image’s barycentre:

$$v_i = b - p_i,$$

where $b = (W/2, H/2)$ is the centre of the model image. The set of model features together with their joining vectors is what is known as a *star model*, and it replaces the R-Table used in the classical Generalised Hough Transform. At test time, every match between model keypoint i and scene keypoint j at position p_j casts a vote for the model’s barycentre location in the scene:

$$\hat{b}_{ij} = p_j + \frac{\sigma_j}{\sigma_i} v_i,$$

where σ_i and σ_j are the SIFT keypoint sizes. The ratio σ_j/σ_i absorbs the change in apparent scale between model and scene, so the same product is detected whether it appears at the front or the back of the shelf.

This variant was chosen over the classical R-Table because the front-end already produces rich, scale- and orientation-aware SIFT features. Storing per-feature offsets to the barycentre is both more expressive and a closer fit to what the front-end provides than rebuilding an edge-based R-Table on top of SIFT keypoints.

3.2 Peak detection and bounding boxes

Votes are accumulated into a 2-D histogram over the scene, binned at 10 px per cell. Local maxima are extracted by comparing the accumulator against a max-filtered copy (`cv2.dilate`). A bin is a peak if its value equals the local max and exceeds 8 votes. `cv2.dilate` is used for the peak search because it gives an exact local-max test in a single vectorised pass, which is more efficient than scanning the accumulator by hand and equivalent in outcome.

Within a suppression radius of 150 px around each peak the matches that voted for it are gathered. The median of their scale ratios σ_j/σ_i is taken, and a bounding box of $W_{\text{model}} \times \text{median scale}$ by $H_{\text{model}} \times \text{median scale}$ is produced, centred on the peak. The median is used rather than the mean because it survives a handful of outlier voters cleanly, which an arithmetic mean does not.

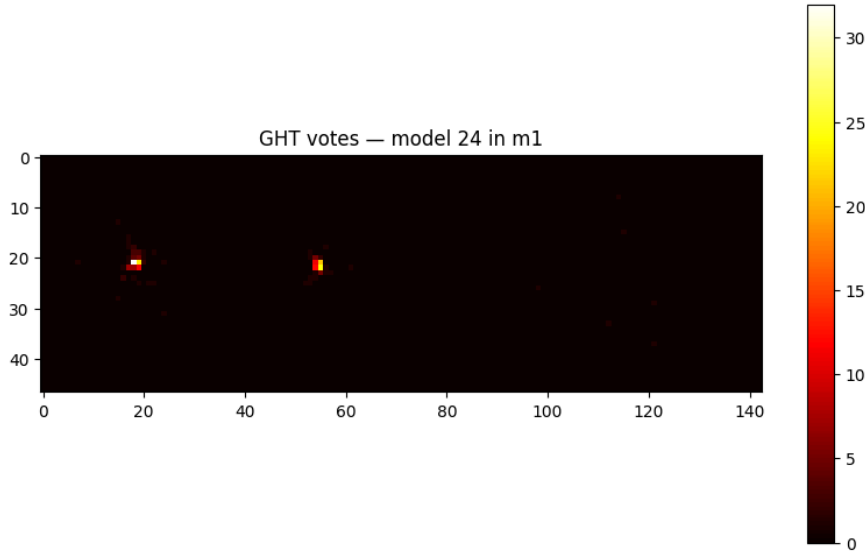


Figure 8: GHT accumulator for product 24 in scene m1. Two clear local maxima correspond to the two box instances.

3.3 Results

The figures below show the accepted detections on each of the five medium shelves after voting, the colour check, and the iterative masking sweep. Green rectangles mark each detected instance's bounding box. The product label is drawn at the centre.



Figure 9: Task B — accepted detections on shelf m1.



Figure 10: Task B — accepted detections on shelf m2.



Figure 11: Task B — accepted detections on shelf m3.



Figure 12: Task B — accepted detections on shelf m4.



Figure 13: Task B — accepted detections on shelf m5.

3.4 Discussion

The colour check from Task A is reused unchanged here — same Euclidean BGR-mean distance, same threshold of 50. The intuition that a product’s colour palette is intrinsic to the product, not to the scene, carries over. The check has more work to do in Task B because each (model, scene) pair can produce several candidate bounding boxes from the GHT peaks, but no per-scene retuning was needed. How effectively the colour check really separates the products in Task B is not clear or quantified, but the same arguments from Task A apply, and the visual results look good.

The same iterative-masking sweep as Task A wraps the per-pair pipeline, with two adjustments. First, the per-pair stage returns a *list* of accepted instances rather than a single match, one bounding box per GHT peak that survives the colour check. Second, the arbitration score is $\text{len}(\text{accepted}) \times \text{len}(\text{good})$ rather than RANSAC inlier count, since RANSAC is no longer in the loop. When a product wins an iteration, all of its accepted instances are committed to the detection list at once, and their bounding boxes are masked out in the working scene before the next iteration. The product of the two counts is used because either factor alone doesn’t work. Tuning the PEAK_SUPPRESSION_RADIUS was necessary to not overlap peaks from different instances of the same product, specifically p0 and p26. The value of 150 px was found empirically to give good results across all shelves.

4 Conclusion

This project implemented a layered SIFT-based pipeline that covers both single-instance detection (Task A) and multi-instance detection (Task B), sharing a SIFT + FLANN + Lowe front-end and a Euclidean BGR-

mean colour check. The two tasks diverge at the verification stage, where task A uses a single RANSAC homography, while Task B replaces it with a star-model GHT.

Detections are correct on all easy and medium scenes, including shelves with multiple copies of the same product. The biggest limitation and failure observed is the homography on product 24 in scene e4, which is likely caused by the price tag line confusing the geometric verification.

Natural extensions include a per-region HSV or LAB colour histogram in place of a single BGR mean, voting in a 4-D pose space (t_x, t_y, θ, s) rather than only translation. More principled multi-instance detection strategies could be explored, such as non-max suppression on the GHT accumulator or a global inference step that considers all peaks and their supporters together rather than greedily accepting one at a time.

5 References

- [1] L. D. Stefano, “Local features part 1” University of Bologna, Computer Vision and Image Processing, 2025. Accessed: 2026-05-18.
- [2] L. D. Stefano, “Local features part 2” University of Bologna, Computer Vision and Image Processing, 2025. Accessed: 2026-05-18.
- [3] L. D. Stefano, “Instance Detection” University of Bologna, Computer Vision and Image Processing, 2025. Accessed: 2026-05-16.
- [4] O. Documentation, “OpenCV Documentation”, OpenCV Docs, 2026. Accessed: 2026-05-15 [Online]. Available: <https://docs.opencv.org/4.x/>